DESARROLLO DE APLICACIONES CON OBJETOS · 1º PARCIAL

Ejemplos de clase: clases y herencia

El código que se ve en clase, con lo que muestra cada ejemplo y su salida

14 ejemplos | Clases y propiedades | Composición | Herencia y clases abstractas

Qué hay acá

Los **ejemplos que se ven en clase** sobre clases, composición, herencia y clases abstractas, tal cual los da la cátedra, con lo que muestra cada uno y la **salida** al correrlos. Están en el ZIP para probarlos y modificarlos.

Concepto	En Python	Ejemplo
Clase y objeto	`class Persona:` · `p = Persona("Ana", 30)`	Firstclass
Constructor	`def __init__(self, ...)`	Firstclass
Propiedades	`@property` y `@x.setter` (validan al asignar)	Firstclasswithproperties
Composición (“tiene un”)	Un atributo que es otro objeto	Coche, Veterinaria
Herencia (“es un”)	`class Perro(Animal):` + `super().__init__(...)`	Herencia
Clase abstracta	`class Figura(ABC):` + `@abstractmethod`	Abstraccion_*
Polimorfismo	El mismo método, distinto resultado según la clase	Herencia, Abstraccion_descuentos

Clases y objetos

Firstclass Primera clase

Qué muestra

Una clase con `__init__`, atributos y métodos; crear un objeto y usarlo.

Para tener en cuenta

- La línea `persona1.edad = -50` muestra el problema: sin propiedades, se le puede poner cualquier valor a un atributo. El ejemplo siguiente lo resuelve.

Firstclass.py

```python
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print(f"Hola, mi nombre es {self.nombre} y tengo {self.edad} años.")

    def cumpleanios(self):
        self.edad += 1
        print(f"Feliz cumpleaños, ahora tengo {self.edad} años!")

# Crear una instancia de la clase Persona
persona1 = Persona("Juan", 30)
persona1.edad=-50
# Acceder a las propiedades y métodos de la instancia
print(persona1.nombre)  # Acceder a la propiedad nombre
print(persona1.edad)    # Acceder a la propiedad edad
persona1.saludar()      # Llamar al método saludar
persona1.cumpleanios()  # Llamar al método cumpleanios
```

Salida al correrlo

```
Juan
-50
Hola, mi nombre es Juan y tengo -50 años.
Feliz cumpleaños, ahora tengo -49 años!
```

Firstclasswithproperties Clase con propiedades

Qué muestra

`@property` y `@nombre.setter`: el atributo se guarda "privado" (`_nombre`) y el setter valida antes de asignar.

Firstclasswithproperties.py

```python
class Persona:
    def __init__(self, nombre, edad):
        self._nombre = nombre  # Propiedad privada con guion bajo
        self._edad = edad

    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, nuevo_nombre):
        if isinstance(nuevo_nombre, str):
            self._nombre = nuevo_nombre
        else:
            raise ValueError("El nombre debe ser una cadena de caracteres")

    @property
    def edad(self):
        return self._edad

    @edad.setter
    def edad(self, nueva_edad):
        if nueva_edad >= 0:
            self._edad = nueva_edad
        else:
            raise ValueError("La edad debe ser un número positivo")

# Crear una instancia de la clase Persona
persona = Persona("Juan", 30)

# Obtener el valor de la propiedad nombre
print(persona.nombre)

# Establecer un nuevo valor para la propiedad nombre
persona.nombre = "Carlos"
print(persona.nombre)

# Intentar establecer un valor no válido para la propiedad nombre
try:
    persona.nombre = 123  # Esto generará una excepción
except ValueError as e:
    print(e)
```

Salida al correrlo

```
Juan
Carlos
El nombre debe ser una cadena de caracteres
```

Claseconcolecciones Clase con colecciones

Qué muestra

Un objeto que guarda un diccionario (productos → precio) y una lista (clientes), con métodos para agregar y listar.

Claseconcolecciones.py

```
class Tienda:
    def __init__(self, nombre):
        self.nombre = nombre
        self.productos = {}  # Un diccionario para almacenar productos (nombre -> precio)
        self.clientes = []   # Una lista para almacenar los clientes

    def agregar_producto(self, nombre, precio):
        self.productos[nombre] = precio

    def listar_productos(self):
        print(f"Productos en la tienda {self.nombre}:")
        for nombre, precio in self.productos.items():
            print(f"{nombre}: ${precio}")

    def agregar_cliente(self, nombre):
        self.clientes.append(nombre)

    def listar_clientes(self):
        print(f"Clientes de la tienda {self.nombre}:")
        for cliente in self.clientes:
            print(cliente)

# Crear una instancia de la clase Tienda
mi_tienda = Tienda("Mi Tienda")

# Agregar productos
mi_tienda.agregar_producto("Camiseta", 15)
mi_tienda.agregar_producto("Pantalón", 30)
mi_tienda.agregar_producto("Zapatos", 50)

# Listar productos
mi_tienda.listar_productos()

# Agregar clientes
mi_tienda.agregar_cliente("Juan")
mi_tienda.agregar_cliente("Ana")
mi_tienda.agregar_cliente("Carlos")

# Listar clientes
mi_tienda.listar_clientes()
```

Salida al correrlo

```
Productos en la tienda Mi Tienda:
Camiseta: $15
Pantalón: $30
Zapatos: $50
Clientes de la tienda Mi Tienda:
Juan
Ana
Carlos
```

Coche Composición

Qué muestra

El coche **tiene un** motor y **tiene cuatro** ruedas: los crea en su `__init__` y les delega el trabajo.

Coche.py

```python
class Motor:
    def arrancar(self):
        print("Motor arrancado")

class Rueda:
    def girar(self):
        print("Rueda girando")

class Coche:
    def __init__(self):
        self.motor = Motor()  # Composición: el coche "tiene un" motor
        self.ruedas = [Rueda() for _ in range(4)]  # Composición: el coche "tiene cuatro" ruedas

    def conducir(self):
        self.motor.arrancar()
        for rueda in self.ruedas:
            rueda.girar()

# Crear un objeto Coche
mi_coche = Coche()

# Conducir el coche
mi_coche.conducir()
```

Salida al correrlo

```
Motor arrancado
Rueda girando
Rueda girando
Rueda girando
Rueda girando
```

Coche_funciona Composición en dos archivos

Qué muestra

Las mismas clases en `Coche_partes.py` y el programa en otro archivo que las importa (`import Coche_partes`).

Coche_partes.py

```
class Motor:
    def arrancar(self):
        print("Motor arrancado")

class Rueda:
    def girar(self):
        print("Rueda girando")

class Coche:
    def __init__(self):
        self.motor = Motor()  # Composición: el coche "tiene un" motor
        self.ruedas = [Rueda() for _ in range(4)]  # Composición: el coche "tiene cuatro" ruedas

    def conducir(self):
        self.motor.arrancar()
        for rueda in self.ruedas:
            rueda.girar()
```

Coche_funciona.py

```
import Coche_partes

# Crear un objeto Coche
mi_coche = Coche_partes.Coche()

# Conducir el coche
mi_coche.conducir()
```

Salida al correrlo

```
Motor arrancado
Rueda girando
Rueda girando
Rueda girando
Rueda girando
```

Persona_pantalla Personas cargadas por teclado

Qué muestra

Crear objetos con datos que ingresa el usuario, guardarlos en una lista y buscar el de menor edad.

Persona_pantalla.py

```
class Persona:
    def __init__(self, documento, nombre, apellido, edad):
        self.documento = documento
        self.nombre = nombre
        self.apellido = apellido
        self.edad = edad

# Crear una lista para almacenar las instancias de Persona
personas = []

# Solicitar al usuario la cantidad de personas a ingresar
cantidad_personas = int(input("Ingrese la cantidad de personas a registrar: "))

# Pedir datos de las personas y crear instancias de Persona
for i in range(cantidad_personas):
    print(f"Ingresando datos de la persona {i + 1}:")
    documento = input("Documento: ")
    nombre = input("Nombre: ")
    apellido = input("Apellido: ")
    edad = int(input("Edad: "))
    persona = Persona(documento, nombre, apellido, edad)
    personas.append(persona)

# Mostrar el estado de las personas ingresadas
print("\nEstado de las personas ingresadas:")
for persona in personas:
    print(f"Documento: {persona.documento}")
    print(f"Nombre: {persona.nombre}")
    print(f"Apellido: {persona.apellido}")
    print(f"Edad: {persona.edad}")
    print("-" * 20)

# Encontrar y mostrar la persona de menor edad
persona_menor_edad = min(personas, key=lambda x: x.edad)
print(f"La persona de menor edad es: {persona_menor_edad.nombre} {persona_menor_edad.apellido}
(Edad: {persona_menor_edad.edad})")
```

Salida al correrlo (con lo que se tipeó)

```
Ingrese la cantidad de personas a registrar: 2
Ingresando datos de la persona 1:
Documento: 30111222
Nombre: Ana
Apellido: López
Edad: 25
Ingresando datos de la persona 2:
Documento: 40123456
Nombre: Sofía
Apellido: Gómez
Edad: 19

Estado de las personas ingresadas:
Documento: 30111222
Nombre: Ana
Apellido: López
Edad: 25
--------------------
Documento: 40123456
Nombre: Sofía
Apellido: Gómez
Edad: 19
--------------------
La persona de menor edad es: Sofía Gómez (Edad: 19)
```

Persona_archivo Personas desde un archivo

Qué muestra

Leer `personas.csv` (2000 personas), crear un objeto por línea y hacer consultas sobre la lista.

Para tener en cuenta

- El mismo archivo venía en los dos ZIP de ejemplos: está una sola vez.

Persona_archivo.py

```python
import csv

class Persona:
    def __init__(self, documento, nombre, apellido, edad):
        self.documento = documento
        self.nombre = nombre
        self.apellido = apellido
        self.edad = int(edad)

# Crear un diccionario para almacenar las instancias de Persona indexadas por documento
personas_dict = {}

# Leer el archivo personas.csv y crear instancias de Persona
with open('personas.csv', 'r') as archivo_csv:
    lector = csv.reader(archivo_csv)
    for linea in lector:
        documento, nombre, apellido, edad = linea
        persona = Persona(documento, nombre, apellido, edad)
        personas_dict[documento] = persona

# Informe 1: Cantidad de personas cargadas
cantidad_personas = len(personas_dict)
print(f"Cantidad de personas cargadas: {cantidad_personas}")

# Informe 2: Cantidad de personas mayores de edad
mayores_de_edad = [persona for persona in personas_dict.values() if persona.edad >= 18]
cantidad_mayores_de_edad = len(mayores_de_edad)
print(f"Cantidad de personas mayores de edad: {cantidad_mayores_de_edad}")

# Informe 3: Listado de nombres y apellidos de personas cuyo apellido empiece en vocal
apellidos_con_vocal = [persona for persona in personas_dict.values() if
persona.apellido[0].lower() in 'aeiou']
print("Personas cuyo apellido empieza en vocal:")
for persona in apellidos_con_vocal:
    print(f"{persona.nombre} {persona.apellido}")

# Informe 4: Cantidad de personas por cada apellido
apellidos = [persona.apellido for persona in personas_dict.values()]
cantidad_por_apellido = {apellido: apellidos.count(apellido) for apellido in set(apellidos)}
print("Cantidad de personas por cada apellido:")
for apellido, cantidad in cantidad_por_apellido.items():
    print(f"{apellido}: {cantidad}")
```

Salida al correrlo

```
Cantidad de personas cargadas: 2000
Cantidad de personas mayores de edad: 1815
Personas cuyo apellido empieza en vocal:
IVAN ALVAREZ
MARIA CARMEN AGUILAR
MARC ORTIZ
JUAN CARLOS ORTIZ
TOMAS ORTEGA
MIGUEL AGUILAR
ANA ORTIZ
JOSE ANGEL ALONSO
CAROLINA ESTEBAN
MOHAMED ALONSO
IGNACIO AGUILAR
MARGARITA ALVAREZ
JESUS IGLESIAS
ROSARIO IBAÑEZ
MARIANO IGLESIAS
ANDREA ORTIZ
ROBERTO ALONSO
JUAN LUIS ORTIZ
ANDRES IGLESIAS
LUISA AGUILAR
MARIA LUISA ALVAREZ
JUAN ANTONIO AGUILAR
RUBEN ALVAREZ
… (260 renglones más)
REYES: 21
IBAÑEZ: 20
ROMERO: 26
LORENZO: 23
CABALLERO: 15
RODRIGUEZ: 20
```

Herencia, abstracción e interfaces

Herencia Herencia

Qué muestra

`Perro` y `Gato` heredan de `Animal` (`super().__init__`) y cada uno redefine `hacer_sonido`: **polimorfismo**.

Herencia.py

```python
# Clase base (superclase)
class Animal:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def hacer_sonido(self):
        pass

# Clase derivada (subclase)
class Perro(Animal):
    def hacer_sonido(self):
        return "Woof!"

# Clase derivada (subclase)
class Gato(Animal):
    def hacer_sonido(self):
        return "Meow!"

# Crear instancias de las subclases
mi_perro = Perro("Rex", 3)
mi_gato = Gato("Whiskers", 2)

# Acceder a los atributos de la superclase
print(f"{mi_perro.nombre} tiene {mi_perro.edad} años.")
print(f"{mi_gato.nombre} tiene {mi_gato.edad} años.")

# Llamar a los métodos de las subclases
print(f"{mi_perro.nombre} hace: {mi_perro.hacer_sonido()}")
print(f"{mi_gato.nombre} hace: {mi_gato.hacer_sonido()}")
```

Salida al correrlo

```
Rex tiene 3 años.
Whiskers tiene 2 años.
Rex hace: Woof!
Whiskers hace: Meow!
```

Interfaces Interfaz “informal”

Qué muestra

Una clase base con un método vacío que las hijas redefinen.

Para tener en cuenta

- Así nada obliga a la hija a implementarlo: si se olvida, hereda el `pass` y devuelve `None` sin avisar.

Interfaces.py

```python
class MiInterfaz:
    def metodo_abstracto(self):
        pass

class MiClaseConcreta(MiInterfaz):
    def metodo_abstracto(self):
        return "Implementación del método abstracto"

pruebaclase=MiClaseConcreta()
print(pruebaclase.metodo_abstracto())
```

Salida al correrlo

```
Implementación del método abstracto
```

Interfacesabstractas Interfaz con ABC

Qué muestra

Lo mismo con `ABC` y `@abstractmethod`: ahora Python **obliga** a la hija a implementar el método (si no, no deja crear objetos).

Interfacesabstractas.py

```python
from abc import ABC, abstractmethod

class MiInterfaz(ABC):  # Importa ABC aquí
    @abstractmethod
    def metodo_abstracto(self):
        pass

class MiClaseConcreta(MiInterfaz):
    def metodo_abstracto(self):
        return "Implementación del método abstracto 2"

claseprueba = MiClaseConcreta()
print(claseprueba.metodo_abstracto())
```

Salida al correrlo

```
Implementación del método abstracto 2
```

Abstraccion_figuras Clase abstracta: figuras

Qué muestra

Cada figura calcula su área a su manera. El ejemplo de la cátedra solo define las clases: abajo hay una prueba que las usa y muestra que la clase abstracta no se puede instanciar.

Abstraccion_figuras.py

```python
from abc import ABC, abstractmethod

class FiguraGeometrica(ABC):
    @abstractmethod
    def calcular_area(self):
        pass

class Cuadrado(FiguraGeometrica):
    def __init__(self, lado):
        self.lado = lado

    def calcular_area(self):
        return self.lado ** 2

class Circulo(FiguraGeometrica):
    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return 3.14159 * self.radio ** 2
```

demo_figuras.py

```python
# Prueba de Abstraccion_figuras.py (el ejemplo de la cátedra solo define las clases)
from Abstraccion_figuras import FiguraGeometrica, Cuadrado, Circulo

print("Área del cuadrado de lado 3:", Cuadrado(3).calcular_area())
print("Área del círculo de radio 2:", Circulo(2).calcular_area())
try:
    FiguraGeometrica()          # una clase abstracta no se puede instanciar
except TypeError as error:
    print("No se puede crear una FiguraGeometrica:", error)
```

Salida de demo_figuras.py

```
Área del cuadrado de lado 3: 9
Área del círculo de radio 2: 12.56636
No se puede crear una FiguraGeometrica: Can't instantiate abstract class FiguraGeometrica with
abstract method calcular_area
```

Abstraccion_vehiculos Clase abstracta: vehículos

Qué muestra

Una clase abstracta con atributos comunes (marca, modelo, año) y métodos abstractos que cada vehículo implementa.

Abstraccion_vehiculos.py

```python
from abc import ABC, abstractmethod

class Vehiculo(ABC):
    def __init__(self, marca, modelo, año):
        self.marca = marca
        self.modelo = modelo
        self.año = año

    @abstractmethod
    def arrancar(self):
        pass

    @abstractmethod
    def detener(self):
        pass

    def obtener_informacion(self):
        return f"{self.año} {self.marca} {self.modelo}"

class Coche(Vehiculo):
    def arrancar(self):
        return f"El coche {self.obtener_informacion()} ha arrancado."

    def detener(self):
        return f"El coche {self.obtener_informacion()} se ha detenido."

class Moto(Vehiculo):
    def arrancar(self):
        return f"La moto {self.obtener_informacion()} ha arrancado."

    def detener(self):
        return f"La moto {self.obtener_informacion()} se ha detenido."

# Crear instancias de las clases derivadas
mi_coche = Coche("Toyota", "Corolla", 2022)
mi_moto = Moto("Honda", "CBR 600", 2021)

# Utilizar los métodos de las clases derivadas
print(mi_coche.arrancar())  # Salida: El coche 2022 Toyota Corolla ha arrancado.
print(mi_moto.detener())    # Salida: La moto 2021 Honda CBR 600 se ha detenido.
```

Salida al correrlo

```
El coche 2022 Toyota Corolla ha arrancado.
La moto 2021 Honda CBR 600 se ha detenido.
```

Abstraccion_descuentos Clase abstracta: descuentos

Qué muestra

Cada tipo de cliente calcula su descuento: el mismo mensaje da resultados distintos según la clase.

Abstraccion_descuentos.py

```python
from abc import ABC, abstractmethod

# Clase abstracta Cliente
class Cliente(ABC):
    def __init__(self, nombre, monto_compra):
        self.nombre = nombre
        self.monto_compra = monto_compra

    @abstractmethod
    def calcular_descuento(self):
        pass

# Subclase ClienteRegular
class ClienteRegular(Cliente):
    def calcular_descuento(self):
        if self.monto_compra > 100:
            return self.monto_compra * 0.05  # Descuento del 5%
        else:
            return 0

# Subclase ClientePremium
class ClientePremium(Cliente):
    def calcular_descuento(self):
        return self.monto_compra * 0.10  # Descuento del 10%

# Función para calcular el monto total después del descuento
def calcular_total_con_descuento(cliente):
    descuento = cliente.calcular_descuento()
    total = cliente.monto_compra - descuento
    return total

# Crear instancias de clientes
cliente1 = ClienteRegular("Cliente Regular 1", 120)
cliente2 = ClientePremium("Cliente Premium 1", 150)

# Calcular el monto total con descuento para cada cliente
total_cliente1 = calcular_total_con_descuento(cliente1)
total_cliente2 = calcular_total_con_descuento(cliente2)

# Mostrar resultados
print(f"{cliente1.nombre}: Monto a pagar después del descuento: ${total_cliente1}")
print(f"{cliente2.nombre}: Monto a pagar después del descuento: ${total_cliente2}")
```

Salida al correrlo

```
Cliente Regular 1: Monto a pagar después del descuento: $114.0
Cliente Premium 1: Monto a pagar después del descuento: $135.0
```

Veterinaria Veterinaria

Qué muestra

Todo junto: clientes que **tienen una** mascota (composición), propiedades y consultas sobre la lista.

Veterinaria.py

```
class Mascota:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

class Cliente:
    def __init__(self, numero_cliente, nombre, antiguedad, mascota):
        self.numero_cliente = numero_cliente
        self.nombre = nombre
        self.antiguedad = antiguedad
        self.mascota = mascota

# Crear una lista para almacenar los datos de los clientes
clientes = []

# Solicitar al usuario la cantidad de clientes a ingresar
cantidad_clientes = int(input("Ingrese la cantidad de clientes a registrar: "))

# Pedir datos de los clientes y sus mascotas y crear instancias
for i in range(cantidad_clientes):
    print(f"Ingresando datos del cliente {i + 1}:")
    numero_cliente = int(input("Número de cliente: "))
    nombre_cliente = input("Nombre del cliente: ")
    antiguedad = int(input("Antigüedad (años): "))

    print(f"Ingresando datos de la mascota de {nombre_cliente}:")
    nombre_mascota = input("Nombre de la mascota: ")
    edad_mascota = int(input("Edad de la mascota: "))

    mascota = Mascota(nombre_mascota, edad_mascota)
    cliente = Cliente(numero_cliente, nombre_cliente, antiguedad, mascota)
    clientes.append(cliente)

# Informe 1: Cantidad de clientes
cantidad_total_clientes = len(clientes)
print(f"\nCantidad total de clientes: {cantidad_total_clientes}")

# Informe 2: Promedio de edad de las mascotas
suma_edades_mascotas = sum(cliente.mascota.edad for cliente in clientes)
promedio_edad_mascotas = suma_edades_mascotas / cantidad_total_clientes
print(f"Promedio de edad de las mascotas: {promedio_edad_mascotas:.2f} años")

# Informe 3: Clientes con antigüedad mayor o igual a 5 años
clientes_antiguedad_5_anios = [cliente for cliente in clientes if cliente.antiguedad >= 5]
cantidad_clientes_antiguedad_5_anios = len(clientes_antiguedad_5_anios)
print(f"Cantidad de clientes con antigüedad mayor o igual a 5 años:
{cantidad_clientes_antiguedad_5_anios}")

# Informe 4: Clientes con mascotas mayores de 5 años
clientes_mascotas_mayor_5_anios = [cliente for cliente in clientes if cliente.mascota.edad > 5]
```

```python
print("\nClientes cuyas mascotas tienen más de 5 años:")
for cliente in clientes_mascotas_mayor_5_anios:
    print(f"Nombre del cliente: {cliente.nombre}")
    print(f"Antigüedad del cliente: {cliente.antiguedad} años")
    print(f"Nombre de la mascota: {cliente.mascota.nombre}")
    print(f"Edad de la mascota: {cliente.mascota.edad} años")
    print("-" * 20)
```

Salida al correrlo (con lo que se tipeó)

```
Ingrese la cantidad de clientes a registrar: 2
Ingresando datos del cliente 1:
Número de cliente: 1
Nombre del cliente: Marta
Antigüedad (años): 6
Ingresando datos de la mascota de Marta:
Nombre de la mascota: Toby
Edad de la mascota: 8
Ingresando datos del cliente 2:
Número de cliente: 2
Nombre del cliente: Pablo
Antigüedad (años): 2
Ingresando datos de la mascota de Pablo:
Nombre de la mascota: Luna
Edad de la mascota: 3

Cantidad total de clientes: 2
Promedio de edad de las mascotas: 5.50 años
Cantidad de clientes con antigüedad mayor o igual a 5 años: 1

Clientes cuyas mascotas tienen más de 5 años:
Nombre del cliente: Marta
Antigüedad del cliente: 6 años
Nombre de la mascota: Toby
Edad de la mascota: 8 años
--------------------
```